

JDBC Resources, Exceptions and Metadata

Change the URL, username and password to match your database.

Q1: PlaylistFetcher.java (try-with-resources)

```
CREATE TABLE playlists (
    playlist_id INT PRIMARY KEY AUTO_INCREMENT,
    playlist_name VARCHAR(100)
);

// PlaylistFetcher.java
// Retrieves all playlist names. Connection, Statement and ResultSet are closed
// automatically by try-with-resources.

import java.sql.*;

public class PlaylistFetcher {
    private static final String URL = "jdbc:mysql://localhost:3306/spotify_clone";
    private static final String USER = "root";
    private static final String PASS = "root123";

    public static void main(String[] args) {
        String sql = "SELECT playlist_name FROM playlists";

        try (Connection con = DriverManager.getConnection(URL, USER, PASS);
            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery(sql)) {

            System.out.println("Playlist Names:");
            while (rs.next()) {
                System.out.println("- " + rs.getString("playlist_name"));
            }

        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
        // No finally block needed: all three resources are already closed here.
    }
}
```

Sample output:

```
Playlist Names:
- Workout Mix
- Chill Vibes
- Bollywood Hits
```

Why it is safe: resources declared in the try(...) parentheses are closed automatically in reverse order (ResultSet, Statement, Connection), even if an exception is thrown.

Q2: Handling SQL exceptions with a friendly message

Original code (poor error handling):

```
public static void main(String[] args) throws Exception {
    Connection con = DriverManager.getConnection(URL, USER, PASS);
    Statement st = con.createStatement();
    ResultSet rs = st.executeQuery("SELECT username, email FROM users");
    while (rs.next()) {
        System.out.println(rs.getString("username") + " - " + rs.getString("email"));
    }
}
// If the connection fails, the program crashes and shows a long stack trace.
```

Modified code:

```
// UserFetcher.java
// Fetches user data and shows a user-friendly message if the database connection fails.
```

```

import java.sql.*;

public class UserFetcher {
    private static final String URL = "jdbc:mysql://localhost:3306/myapp_db";
    private static final String USER = "root";
    private static final String PASS = "root123";

    public static void main(String[] args) {
        String sql = "SELECT username, email FROM users";

        try (Connection con = DriverManager.getConnection(URL, USER, PASS);
            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery(sql)) {

            while (rs.next()) {
                System.out.println(rs.getString("username") + " - " + rs.getString("email"));
            }

        } catch (SQLException e) {
            // Custom messages instead of the stack trace
            if (e.getErrorCode() == 1045) {
                System.out.println("Login failed: the database username or password is incorrect.");
            } else if (e.getErrorCode() == 1049) {
                System.out.println("Database not found: please check the database name.");
            } else if (e.getSQLState() != null && e.getSQLState().startsWith("08")) {
                System.out.println("Could not connect to the database. Please make sure MySQL is running "
                    + "and try again.");
            } else {
                System.out.println("Sorry, something went wrong while fetching users. Please try again
later.");
            }
        }
    }
}

```

Output when MySQL is not running:

Could not connect to the database. Please make sure MySQL is running and try again.

Note: error code 1045 means access denied and 1049 means unknown database. SQLState values starting with "08" mean a connection problem. The error code and SQLState values can vary a little between driver versions.

Q3: ResultSetMetaData for the products table

```

CREATE TABLE products (
    product_id INT PRIMARY KEY AUTO_INCREMENT,
    name        VARCHAR(100),
    price       DECIMAL(10,2),
    category    VARCHAR(50)
);

// ProductMetaDataDemo.java
// Prints the column names and data types of the products table using ResultSetMetaData.

import java.sql.*;

public class ProductMetaDataDemo {
    private static final String URL = "jdbc:mysql://localhost:3306/shop_db";
    private static final String USER = "root";
    private static final String PASS = "root123";

    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS);
            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery("SELECT * FROM products")) {

            ResultSetMetaData meta = rs.getMetaData();
            int columnCount = meta.getColumnCount();

            System.out.println("Total columns: " + columnCount);
            System.out.printf("%-5s %-15s %-12s%n", "No.", "Column Name", "Data Type");

```

```

        System.out.println("-----");

        for (int i = 1; i <= columnCount; i++) { // column numbers start at 1
            System.out.printf("%-5d %-15s %-12s\n",
                i, meta.getColumnName(i), meta.getColumnTypeName(i));
        }

    } catch (SQLException e) {
        System.out.println("Database error: " + e.getMessage());
    }
}
}

```

Output:

```

Total columns: 4
No.   Column Name      Data Type
-----
1     product_id       INT
2     name              VARCHAR
3     price             DECIMAL
4     category          VARCHAR

```

Q4: DatabaseMetaData to list all tables

```

// ListTables.java
// Lists all table names in the current database using DatabaseMetaData.

import java.sql.*;

public class ListTables {
    private static final String URL = "jdbc:mysql://localhost:3306/zomato_db";
    private static final String USER = "root";
    private static final String PASS = "root123";

    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)) {

            DatabaseMetaData dbMeta = con.getMetaData();
            String currentDb = con.getCatalog(); // in MySQL the "catalog" is the database name

            System.out.println("Tables in database: " + currentDb);

            try (ResultSet tables = dbMeta.getTables(currentDb, null, "%", new String[] {"TABLE"})) {
                while (tables.next()) {
                    System.out.println("- " + tables.getString("TABLE_NAME"));
                }
            }

        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }
}

```

Sample output:

```

Tables in database: zomato_db
- customers
- menu_items
- orders
- restaurants

```

getTables(catalog, schemaPattern, tableNamePattern, types) returns one row per table.
 "%" matches all names, and {"TABLE"} excludes views.

Q5: AI-generated JDBC template for inserting a movie

I cannot use GitHub Copilot for you, and your teacher will expect your own screenshot. Do it yourself like this:

Step 1: Paste this prompt into Copilot (or ChatGPT):

"Generate a JDBC template in Java to insert a new record into a MySQL table movies (id, title, genre) using PreparedStatement."

Step 2: Take a screenshot of the prompt and the generated code.

Table:

```
CREATE TABLE movies (  
    id    INT PRIMARY KEY,  
    title VARCHAR(100),  
    genre VARCHAR(50)  
);
```

Step 3: Typical generated template (basic version):

```
Connection con = DriverManager.getConnection(url, user, password);  
PreparedStatement ps = con.prepareStatement("INSERT INTO movies (id, title, genre) VALUES (?, ?, ?)");  
ps.setInt(1, 1);  
ps.setString(2, "Inception");  
ps.setString(3, "Sci-Fi");  
ps.executeUpdate();  
// No exception handling and the resources are never closed
```

Step 4: Final customized code (submit this):

```
// InsertMovie.java  
// Inserts a new movie using PreparedStatement, try-with-resources and exception handling.  
  
import java.sql.*;  
  
public class InsertMovie {  
    private static final String URL = "jdbc:mysql://localhost:3306/bookmyshow_db";  
    private static final String USER = "root";  
    private static final String PASS = "root123";  
  
    public static void insertMovie(int id, String title, String genre) {  
        String sql = "INSERT INTO movies (id, title, genre) VALUES (?, ?, ?)";  
  
        try (Connection con = DriverManager.getConnection(URL, USER, PASS);  
            PreparedStatement ps = con.prepareStatement(sql)) {  
  
            ps.setInt(1, id);  
            ps.setString(2, title);  
            ps.setString(3, genre);  
  
            int rows = ps.executeUpdate();  
            if (rows > 0) {  
                System.out.println("Movie inserted successfully.");  
            }  
  
        } catch (SQLIntegrityConstraintViolationException e) {  
            System.out.println("A movie with ID " + id + " already exists.");  
        } catch (SQLException e) {  
            System.out.println("Database error: " + e.getMessage());  
        }  
    }  
  
    public static void main(String[] args) {  
        insertMovie(1, "Inception", "Sci-Fi");  
    }  
}
```

Output:

Movie inserted successfully.

Customizations to mention:

- Wrapped Connection and PreparedStatement in try-with-resources so they close automatically.
- Added a specific catch for duplicate IDs (SQLIntegrityConstraintViolationException) and a general catch for SQLException.

- Moved the code into a reusable method `insertMovie(id, title, genre)` and added a success message.